

BÀI THỰC HÀNH

LAB 03

Cơ bản về Serial Monitor

Giám sát trạng thái hệ thống với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Truyền thông nối tiếp và Serial Monitor	2
2.2	UART và truyền thông bất đồng bộ	2
2.3	Baud rate	3
2.4	Cấu trúc cơ bản của UART frame	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Giao diện sử dụng	4
4	EduFramework API	4
4.1	<code>Serial1_begin()</code>	4
4.2	<code>Serial1_print()</code>	5
4.3	<code>Serial1_println()</code>	5
5	Bài thực hành	6
5.1	Yêu cầu	6
5.2	Chương trình	6
5.3	Kiểm chứng	7
6	Mở rộng	7
6.1	In giá trị số nguyên	7
6.2	In giá trị số thực	7
6.3	Bài tập mở rộng: đếm số lần nhấn nút	8
6.4	<code>Serial1</code> và <code>Serial2</code>	8
7	Tài liệu tham khảo	9

1 Giới thiệu

1.1 Tổng quan bài lab

Trong quá trình phát triển hệ thống nhúng, việc quan sát trạng thái bên trong chương trình là cần thiết để kiểm chứng hoạt động của phần mềm và theo dõi dữ liệu trong thời gian chạy. Một phương pháp phổ biến là truyền thông tin từ vi điều khiển tới máy tính thông qua giao tiếp nối tiếp và hiển thị dữ liệu trên Serial Monitor.

UART (Universal Asynchronous Receiver/Transmitter) là một cơ chế truyền thông nối tiếp bất đồng bộ thường được tích hợp trong vi điều khiển. Dữ liệu được truyền tuần tự theo từng bit giữa các thiết bị và không yêu cầu một đường clock chung cho quá trình truyền nhận [1].

Bài lab này được thiết kế để giới thiệu Serial Monitor thông qua giao diện `Serial1` của EduFramework. Trạng thái của nút nhấn tích hợp được truyền tới máy tính để quan sát trực tiếp trên Serial Monitor. Qua đó, Serial Monitor trở thành một công cụ cơ bản có thể được tái sử dụng trong các bài lab tiếp theo để quan sát giá trị cảm biến, trạng thái hệ thống và kết quả xử lý.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Giải thích nguyên lý cơ bản của truyền thông nối tiếp bất đồng bộ và vai trò của UART trong việc truyền dữ liệu.
2. Mô tả vai trò của TX, RX, baud rate và cấu trúc cơ bản của một UART frame.
3. Sử dụng các API `Serial1_begin()`, `Serial1_print()` và `Serial1_println()` để truyền dữ liệu tới Serial Monitor.
4. Xây dựng và kiểm chứng ứng dụng giám sát sự kiện nút nhấn trên Serial Monitor.

2 Kiến thức nền

2.1 Truyền thông nối tiếp và Serial Monitor

Trong truyền thông nối tiếp, dữ liệu được truyền tuần tự theo từng bit thay vì truyền đồng thời nhiều bit trên nhiều đường tín hiệu. Phương pháp này được sử dụng rộng rãi trong hệ thống nhúng để trao đổi dữ liệu giữa vi điều khiển và các thiết bị khác [1].

Serial Monitor là một công cụ hiển thị dữ liệu được truyền qua cổng serial. Trong bài lab này, Serial Monitor được sử dụng như một kênh quan sát từ MaaZEDU Development Board tới máy tính. Chương trình có thể gửi các chuỗi ký tự mô tả trạng thái hoặc sự kiện để hỗ trợ quá trình kiểm chứng hoạt động.

2.2 UART và truyền thông bất đồng bộ

UART thực hiện truyền thông nối tiếp bất đồng bộ. Khác với giao tiếp đồng bộ, hai phía không sử dụng một đường clock chung để xác định thời điểm truyền và nhận dữ liệu. Thay vào đó, các thiết bị cần thống nhất các tham số truyền thông, đặc biệt là tốc độ truyền [2].

Một giao tiếp UART thông thường sử dụng hai hướng tín hiệu:

Tín hiệu	Tên	Chức năng
TX	Transmit	Truyền dữ liệu từ thiết bị hiện tại tới thiết bị nhận.
RX	Receive	Nhận dữ liệu được truyền từ thiết bị khác.

Bảng 3.2.1. Hai hướng tín hiệu cơ bản của UART

Bài lab chỉ sử dụng hướng truyền dữ liệu từ S32K144 tới máy tính. Vì vậy, nội dung chính tập trung vào quá trình xuất dữ liệu qua `Serial1`; chức năng nhận dữ liệu sẽ được giới thiệu trong một bài lab sau.

2.3 Baud rate

Baud rate biểu thị tốc độ ký hiệu của giao tiếp. Trong UART nhị phân thông thường, mỗi ký hiệu tương ứng với một bit; vì vậy baud rate xác định khoảng thời gian truyền của từng bit. Hai phía giao tiếp cần sử dụng cấu hình tương thích để dữ liệu được diễn giải chính xác [1].

Ví dụ, khi chương trình khởi tạo:

```
Serial1_begin(9600U);
```

giao diện `Serial1` được cấu hình hoạt động với baud rate `9600`. Serial Monitor trên máy tính cũng cần sử dụng cùng baud rate.

Ghi chú

Với cấu hình clock mặc định hiện tại của EduFramework, giao tiếp Serial nên được sử dụng ở các baud rate thấp để bảo đảm hoạt động ổn định. Trong series bài lab này, `9600 baud` được khuyến nghị và được sử dụng làm cấu hình mặc định cho Serial Monitor. Đây là giới hạn liên quan tới cấu hình hiện tại của framework, không phải giới hạn chung của peripheral LPUART trên S32K144.

2.4 Cấu trúc cơ bản của UART frame

Do UART không sử dụng một đường clock chung, dữ liệu cần được tổ chức theo một frame để phía nhận có thể xác định thời điểm bắt đầu và kết thúc của dữ liệu. Một UART frame cơ bản gồm start bit, các data bit và stop bit; parity có thể được bổ sung tùy theo cấu hình [2].

Một cấu hình phổ biến là `8N1`, gồm:

Thành phần	Ý nghĩa
1 Start bit	Đánh dấu thời điểm bắt đầu một frame dữ liệu.
8 Data bits	Chứa tám bit dữ liệu cần truyền.
No parity	Không sử dụng bit parity.
1 Stop bit	Đánh dấu kết thúc frame trước khi dữ liệu tiếp theo được truyền.

Bảng 3.2.2. Cấu trúc khái quát của cấu hình UART 8N1

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng nút nhấn tích hợp trên MaaZEDU Development Board và Serial Monitor của PlatformIO, do đó không yêu cầu lắp thêm linh kiện hoặc mạch ngoài.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
USB Cable	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 3.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Giao diện sử dụng

Nút nhấn `BTN0` được sử dụng để tạo sự kiện cần giám sát. EduFramework ánh xạ `BTN0` tới chân `PTC12` của S32K144 [5].

Thành phần	Logical Pin	MCU Pin	Chức năng
Nút nhấn tích hợp	<code>BTN0</code>	<code>PTC12</code>	Digital Input

Bảng 3.3.2. Ánh xạ Digital Input sử dụng trong bài thực hành

Đối với dữ liệu Serial, EduFramework sử dụng `Serial1` cho Serial Monitor và thông tin debug thông qua giao diện debug của board. `Serial1` được ánh xạ tới khối ngoại vi `LPUART1` [4].

Giao diện	Khối ngoại vi	Vai trò trong bài lab
<code>Serial1</code>	<code>LPUART1</code>	Truyền dữ liệu từ S32K144 tới Serial Monitor trên máy tính.

Bảng 3.3.3. Giao diện Serial sử dụng trong bài thực hành

Lường dữ liệu trong bài lab có thể được mô tả theo thứ tự:

`BTN0` → chương trình trên S32K144 → `Serial1` → Serial Monitor.

4 EduFramework API

Bài lab sử dụng ba API cơ bản của `Serial1`: `Serial1_begin()` để khởi tạo giao tiếp, `Serial1_print()` để truyền một chuỗi ký tự và `Serial1_println()` để truyền chuỗi ký tự kèm thao tác kết thúc dòng. Các API thuộc lớp Arduino-style API của EduFramework [4].

4.1 `Serial1_begin()`

`Serial1_begin()` khởi tạo giao diện `Serial1` với baud rate được chỉ định.

`Serial1_begin`

Khởi tạo `Serial1` để truyền và nhận dữ liệu với baud rate được chỉ định.

Cú pháp

```
Serial1_begin(baudRate);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>baudRate</code>	Baud rate	Tốc độ truyền cần sử dụng cho giao tiếp Serial.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
Serial1_begin(9600U);
```

Trong series bài lab này, 9600 baud được sử dụng cho Serial Monitor theo cấu hình EduFramework hiện tại.

4.2 Serial1_print()

`Serial1_print()` truyền một chuỗi ký tự qua `Serial1` mà không tự động kết thúc dòng.

Serial1_print

Truyền chuỗi ký tự được chỉ định qua `Serial1`.

Cú pháp

```
Serial1_print(text);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>text</code>	Chuỗi ký tự	Nội dung cần truyền tới Serial Monitor.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
Serial1_print("System status: ");
```

Dữ liệu được truyền tiếp theo sẽ tiếp tục xuất hiện trên cùng dòng cho đến khi một thao tác kết thúc dòng được thực hiện.

4.3 Serial1_println()

`Serial1_println()` truyền một chuỗi ký tự qua `Serial1` và tự động kết thúc dòng sau chuỗi được truyền [4].

Serial1_println

Truyền chuỗi ký tự được chỉ định qua `Serial1` và kết thúc dòng.

Cú pháp

```
Serial1_println(text);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>text</code>	Chuỗi ký tự	Nội dung cần truyền tới Serial Monitor.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
Serial1_println("System started.");
```

Khác với `Serial1_print()`, dữ liệu được truyền bằng `Serial1_println()` kết thúc tại dòng hiện tại và nội dung tiếp theo được hiển thị trên dòng mới. Cách tổ chức `print()` và `println()` tương ứng cũng được sử dụng trong mô hình Serial của Arduino [3].

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình sử dụng `BTN0` làm nguồn tạo sự kiện và hiển thị trạng thái nút nhấn trên Serial Monitor.

Chương trình cần đáp ứng các yêu cầu sau:

- Khởi tạo `Serial1` với baud rate `9600`.
- Hiển thị thông báo khi hệ thống bắt đầu hoạt động.
- Khi `BTN0` được nhấn, Serial Monitor hiển thị thông báo tương ứng đúng một lần.
- Khi `BTN0` được thả, Serial Monitor hiển thị thông báo tương ứng đúng một lần.
- Giữ nút không làm cùng một thông báo được in liên tục.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình như sau:

```
#include "Arduino.h"

int main(void)
{
    bool pressed = false;

    setup();

    pinMode(BTN0, INPUT);

    Serial1_begin(9600U);

    Serial1_print("EduFramework ");
    Serial1_println("Serial Monitor");
    Serial1_println("System started.");

    while (1)
    {
        if ((HIGH == digitalRead(BTN0)) && (false == pressed))
        {
            Serial1_println("BTN0 pressed.");
            pressed = true;
        }

        if ((LOW == digitalRead(BTN0)) && (true == pressed))
        {
            Serial1_println("BTN0 released.");
            pressed = false;
        }
    }

    return 0;
}
```

Mã nguồn 3.5.1. Chương trình giám sát sự kiện nút nhấn bằng Serial Monitor

Sau khi `setup()` khởi tạo EduFramework, `BTN0` được cấu hình làm Digital Input và `Serial1_begin(9600)` khởi tạo giao tiếp Serial với baud rate 9600.

Hai API `Serial1_print()` và `Serial1_println()` được sử dụng để hiển thị thông tin khởi động. Trong vòng lặp chính, biến `pressed` ghi nhận trạng thái xử lý của nút nhấn. Cơ chế này kế thừa nguyên tắc phát hiện sự kiện đã sử dụng trong Lab 02, nhưng thay vì điều khiển một Digital Output, sự kiện được chuyển thành thông tin quan sát trên Serial Monitor.

Khi `BTN0` chuyển sang trạng thái nhấn, thông báo `BTN0 pressed.` được truyền một lần. Khi nút được thả, chương trình truyền `BTN0 released.` và sẵn sàng xử lý lần nhấn tiếp theo.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board. Sau đó mở Serial Monitor và cấu hình baud rate 9600.

Quan sát nội dung hiển thị khi chương trình bắt đầu, sau đó lần lượt nhấn, giữ và thả `BTN0`.

KẾT QUẢ MONG ĐỢI

Serial Monitor hiển thị thông tin khởi động của hệ thống. Mỗi lần `BTN0` được nhấn, thông báo `BTN0 pressed.` xuất hiện đúng một lần; khi nút được thả, thông báo `BTN0 released.` xuất hiện đúng một lần. Việc giữ nút không làm cùng một thông báo được lặp lại liên tục.

6 Mở rộng

Các API cơ bản trong bài thực hành tập trung vào việc truyền chuỗi ký tự. Trong các ứng dụng đo lường và giám sát, dữ liệu cần quan sát thường còn bao gồm các giá trị số như số lần xảy ra sự kiện, giá trị ADC hoặc kết quả tính toán từ cảm biến. EduFramework cung cấp thêm các API hỗ trợ trực tiếp việc in số nguyên và số thực qua `Serial1` [4].

6.1 In giá trị số nguyên

`Serial1_printInt()` truyền một giá trị số nguyên có dấu qua `Serial1`, trong khi `Serial1_printlnInt()` thực hiện cùng thao tác và kết thúc dòng sau giá trị.

API	Cú pháp	Chức năng
<code>Serial1_printInt()</code>	<code>Serial1_printInt(value);</code>	In một giá trị số nguyên mà không tự động chuyển dòng.
<code>Serial1_printlnInt()</code>	<code>Serial1_printlnInt(value);</code>	In một giá trị số nguyên và kết thúc dòng.

Bảng 3.6.1. API xuất giá trị số nguyên qua `Serial1`

Ví dụ:

```
Serial1_print("Value: ");  
Serial1_printlnInt(value);
```

Cách tách phần mô tả bằng chuỗi ký tự và phần dữ liệu số giúp thông tin trên Serial Monitor dễ đọc hơn.

6.2 In giá trị số thực

Đối với dữ liệu dạng số thực, EduFramework cung cấp `Serial1_printFloat()` và `Serial1_printlnFloat()`. Trong phiên bản hiện tại của EduFramework, giá trị số thực được hiển thị với ba chữ số sau dấu thập phân [4].

API	Cú pháp	Chức năng
<code>Serial1_printFloat()</code>	<code>Serial1_printFloat(value)</code>	In một giá trị số thực mà không tự động chuyển dòng.
<code>Serial1_printlnFloat()</code>	<code>Serial1_printlnFloat(value)</code>	In một giá trị số thực và kết thúc dòng.

Bảng 3.6.2. API xuất giá trị số thực qua Serial1

Ví dụ:

```
Serial1_print("Temperature: ");
Serial1_printFloat(temperature);
Serial1_println(" C");
```

Các API này sẽ được tái sử dụng trong các bài lab liên quan tới ADC và cảm biến, khi giá trị đo cần được quan sát trực tiếp trên Serial Monitor.

6.3 Bài tập mở rộng: đếm số lần nhấn nút

Mở rộng chương trình chính để theo dõi tổng số lần `BTN0` được nhấn kể từ khi hệ thống khởi động. Mỗi sự kiện nhấn hợp lệ chỉ được tăng bộ đếm một lần.

Sau mỗi lần nhấn, Serial Monitor cần hiển thị kết quả theo dạng:

```
BTN0 pressed. Count: 1
BTN0 pressed. Count: 2
BTN0 pressed. Count: 3
```

Sử dụng `Serial1_print()` kết hợp với API xuất số nguyên phù hợp để tạo một dòng thông tin gồm cả chuỗi ký tự và giá trị bộ đếm.

Không thay đổi yêu cầu phát hiện sự kiện của chương trình chính: việc giữ nút không được làm bộ đếm tăng liên tục.

KẾT QUẢ MONG ĐỢI

Mỗi lần `BTN0` được nhấn và được nhận diện như một sự kiện mới, giá trị bộ đếm tăng thêm một. Serial Monitor hiển thị số lần nhấn tích lũy theo đúng thứ tự sự kiện.

6.4 Serial1 và Serial2

EduFramework cung cấp hai giao diện Serial phần cứng với mục đích sử dụng khác nhau [4].

Giao diện	Khối ngoại vi	Mục đích sử dụng
<code>Serial1</code>	<code>LPUART1</code>	Serial Monitor và thông tin debug thông qua giao diện debug của board.
<code>Serial2</code>	<code>LPUART2</code>	Giao tiếp UART với thiết bị hoặc module bên ngoài thông qua các chân UART tương ứng.

Bảng 3.6.3. Vai trò của Serial1 và Serial2 trong EduFramework

Trong các bài lab cần quan sát dữ liệu trên máy tính, `Serial1` được ưu tiên sử dụng làm kênh giám sát. `Serial2` được dành cho các ứng dụng cần trao đổi dữ liệu với thiết bị UART bên ngoài. Nội dung nhận dữ liệu và xử lý lệnh qua UART sẽ được giới thiệu trong một bài lab sau.

7 Tài liệu tham khảo

- [1] Jonathan Valvano and Ramesh Yerraballi, *Chapter 9: Serial Communication* , Introduction to Embedded Systems - The University of Texas at Austin.
- [2] University of Florida, *Lab 5: Asynchronous Serial Communication* , EEL4744C - Electrical & Computer Engineering Department.
- [3] Arduino, *Arduino Language Reference* , Arduino Documentation.
- [4] EduFramework, *Hardware Serial API* , EduFramework Source Code.
- [5] MaaZEDU, *MaaZEDU Development Board Guide* .